



Optimization Techniques for Multi Cooperative Systems MCTR 1021
Mechatronics Engineering
Faculty of Engineering and Materials Science
German University in Cairo

Multi-Robot Path Planning for Dynamic Formation Applications

By

Team 9

**Mona Elbougday
Ahmed Zaghloul
John Gameel
Abdelrahman Saad
Bishoy Atef**

Course Team

**Dr. Omar M. Shehata
M.Sc. Shaimaa Elbaklish**

December 2, 2021

This is to certify that:

- (i) The report comprises only our original work toward the course project.
- (ii) Due acknowledgment has been made in the text to all other material used.

Team 9
December 2, 2021

Acknowledgments

I would like to thank the following for their efforts with me in my study and for tolerating me during this journey...

Abstract

The abstract of the document is added here...
It is usually written after finishing writing all the other chapters.

Contents

Acknowledgments	iii
List of Abbreviations	vii
List of Figures	viii
List of Tables	ix
1 Introduction	1
2 Literature Review	2
3 Problem Formulation	5
3.1 Decision Variables	5
3.2 Objective Functions	6
3.3 Constraints	7
3.4 Mobile Robot Kinematics	7
4 Simulated Annealing	10
4.1 Problem Specific Code	10
4.2 SA Algorithm	10
4.3 Flow Chart	10
5 Genetic Algorithm	12
5.1 Problem Specific Code	12
5.2 GA Algorithm	12
5.3 Flow Chart	13
6 Results	14
6.1 SA	14
6.2 GA	17

7 Conclusion	22
8 Future Work	23
Appendix	24
References	25

List of Abbreviations

List of Figures

3.1	Visual kinematic analysis of the differential mobile robot	8
4.1	Flowchart of SA Algorithm	11
5.1	Flowchart of GA Algorithm	13
6.1	Results of running the SA with linear cooling schedule	15
6.2	Results of running the SA with linear cooling schedule	15
6.3	Results of running the SA with geometric cooling schedule	16
6.4	Results of running the SA with linear cooling schedule	16
6.5	Results of running the GA with Roulette wheel selection	17
6.6	Results of running the GA and visualizing on a map for the Roulette wheel selection	17
6.7	Results of running the GA using Tournament selection	18
6.8	Results of running the GA and visualizing on a map for the Tournament selec- tion	18
6.9	Results of running the GA With Rank selection	19
6.10	Results of running the GA and visualizing on a map for the Rank selection . . .	19
6.11	Results of running the GA and visualizing on a map for the SUS selection . . .	20
6.12	Results of running the GA and visualizing on a map for the Rank SUS selection	20

List of Tables

6.1	Linear Run Parameters	14
6.2	Geometric Run Parameters	16

Chapter 1

Introduction

Multi-robot systems (MRS) employ the idea that there is strength in numbers. The advantages of using multiple robots to perform a certain task become especially noticeable when the task at hand requires multiple simultaneous operations. In this task, the restrictions of a single robot system can be overcome by MRS. In several applications such as exploration, a large number of simple and cheap robots could have better performance than an individual, highly specialized, and costly robot.

Another advantage of MRS is redundancy as tasks can be reallocated to different robots in case of failure of one or more of the robots. This can increase the reliability of the system.

Chapter 2

Literature Review

In [1], the authors provide a distributed flocking control algorithm achieving the three Reynolds's rules; cohesion, alignment, separation. The contribution is minimizing the conflict between cohesion and separation by relaxing the position synchronization constraint. The authors formulate their problem by considering a network of n rigid bodies in 2D Space. They set 2 objective functions: the cohesion and alignment objectives can be considered as pose synchronization and rigid body i and rigid body j are said to collide with each other if the difference between their poses is less than their diameter.

A relaxation parameter δ_i is introduced in the constraints since they aim to achieve position synchronization as much as possible. In simulation, they plot the 3D position trajectories of the rigid body. In results, the proposed flocking algorithm successfully achieves the desired cohesion, alignment, and separation behaviors for the rigid body network.

In [2], a private flocking optimization problem is tackled. The goal is to generate a private flocking that hides the identity of the leader so that an adversary could not identify the leader. The contributions are formulating the problem in homogeneous mobile robot teams with private leaders, developing an adversarial co-optimization method and providing a study of the relationship between flocking and privacy performance.

Two objective functions are set to this problem which are efficient three-dimensional flocking of a team of mobile robots and privacy of this leadership. In the implementation, components of Reynolds's flocking are adopted. Evolutionary approach based on classical genetic algorithms is adopted. Candidate metrics such as velocity correlation accounts for the direction alignment and the magnitude similarity is used. Overall, the findings show that, despite the huge parameter space and growing severity of the adversarial discriminator, the GA is capable of finding extremely effective flocking solutions.

In most flocking protocols each agent is considered to be only collecting information from its neighbors at a certain time, and afterward, it takes motion decisions neglecting the possi-

bility of using predictive intelligence. To make use of individual predictive intelligence among collective behaviors, an MPC has been proposed [3]. The work was evaluated in a simulation where the goal is to minimize the velocity and lattice disagreement. The results suggest that the MPC method was found to have advantages in the convergence efficacy of flocking.

Cooperative control between mobile sensors is essential. Therefore, Flocking control is essential to resolve the problem of self collision. A non-differentiable optimization technique based on the gravitational search algorithm (GSA) was developed in [4]. To evaluate the performance of the proposed method a single target moving with a circular wave, and the multiple targets moving with a semi-circular and semi-sine wave trajectory were simulated. It was found that the center of flocks precisely tracks the moving targets.

The Ant Colony Optimisation (ACO) is one of the mathematically mature optimisation algorithms based on the social behaviour of animal colonies. An ACO Based Hybrid Algorithms was reviewed to be used for Path Planning of Multi-Mobile Robotics in [5]. It was found that The original ACO performs well in complex combinatorial optimisation problems but it has convergence speed and local optima problems. Based on the presented review and survey they were solved by embedding Genetic, A*, Cellular automata, Improved Potential Field and Fuzzy algorithms.

In [6] the authors tackle the problem of flocking control with obstacle avoidance. The mentioned problem is considered a many-objective optimization problem for centralized algorithms. In this work the authors render the many-objective problem into a distributed multi-objective optimization algorithm. There are four objectives for this flocking problem which are: if obstacles are detected pass through them, keep a steady distance with each neighbour, keep a safe distance away from obstacles and avoid collision with neighbours.

The authors divide the four objective into soft constraints (1 and 2) and hard constraints (3 and 4). A modified pigeon-inspired optimization technique is proposed. The problem formulation in this paper starts with a kinematic model for a UAV and sets a number of constraints for horizontal speed and altitude rate. Comparative studies between the Modified MPIO, classic MPIO and NSGA-II were conducted which showed that the modified MPIO is superior.

In [7] the authors develop a flocking technique based on the behavior that pigeons will switch between hierarchical and egalitarian interaction modes at different flight phases. The proposed controller is capable of coordinating the flight of a swarm of UAVs through obstacle environments with few informed individuals. The paper uses a dynamic model with control input that consists of a number of parameters similar to previously discussed work that handle cohesion, alignment and obstacle avoidance. Simulations were conducted and the swarm with very few informed individuals was able to maintain a flock formation and navigate through obstacles. The proposed flocking controller is superior in state convergence time, time reaching target position and trajectory smoothness.

In [8], the authors presented a new flocking model for autonomous flying agents. The four measures of coherence, collision avoidance, obstacle avoidance, and the flocking speed are used to evaluate the performance. They accepted local coherence and correlated sub-flock clusters without global coherence as a good solution. The second measure calculates the number of collisions between the agents by using a characteristic distance above which a collision occurs. The same criteria applies for the obstacle avoidance. The flocking speed is determined by taking the average of the velocities of all agents.

The fitness function used here was resulted from multiplying all fitness functions computed for all the parameters discussed before. The fitness functions used were one of three types, a monotonically growing function, a single maximum at zero and a smooth decay around it or a sharp peak. The decision variables in this optimization problem are 11 variables coming from their new model to achieve the required flocking.

A hybrid flocking controller is presented in [9]. The hybrid flocking control of UAVs is achieved by using three types of guidance methods: vector field, Chucker-Smale model, and potential field. The usage of several methods in hybrid flocking may lead to conflict command generation which degrades the performance of the mission. This issue is addressed by presenting the adaptive Chucker-Smale model. The optimal weightings between guidance methods is determined using social learning particle swarm optimization (SL-PSO). The research uses a dynamic model for his UAV with control inputs that consist of desired speed, heading angle and target point. After which the different guidance methods are formalized. The weighting is done using PSO with 4 different Fitness functions that control cohesion, alignment, velocity matching and obstacle avoidance. The final fitness function is the result of multiplication of the previous fitness functions and tries to maximize it. Simulations were conducted and showed that the proposed adaptive augmented CS model improves the mission performance in comparison to the conventional CD model.

In [10], a new genetic flocking optimizing framework is designed. This framework is able to achieve both stability and adaptivity of robotic swarms. In this work the robot is separated into three layers: sensing, decision and action. These layers support the basic navigation functions. The environment on the other hand is divided into two layers: the evaluation and evolutionary layers where the evaluation layer provides fitness values for the evolutionary one. The researchers propose a velocity update function which take into account Reynolds' definition of flocking maintaining cohesion, separation and alignment. This formulation is used to create a single objective optimization problem through calculating the variation range of an-isotropic index. This allows using GA to optimize this problem. The resulting model is then simulated and compared with basic rule-based model. The new model performs better than the basic rule-based model in uniformity and stability.

Chapter 3

Problem Formulation

In this chapter, we will present the decision variables that we are basing our problem on, the objective function which is the goal that we aspire to reach, and the constraints that our system is subject to them. The problem consists of n nonholonomic differential mobile robots. The robots operate in a continuous map that has a length L and a width W . The map contains a number O of obstacles, each i_{th} obstacle has radius $r_{o \rightarrow i}$. The robots start around point S and their goal is to reach point G while flocking. According to Reynolds, there are three main components of flocking [1]:

- Cohesion
- Separation
- Alignment

To be more specific robots should maintain a close distance between each other (Cohesion), robots also match their velocities (Alignment) and avoid collision among themselves (Separation).

Here we define several variables which are crucial to our formulation:

- r_{bi} is the radius of the i_{th} robot,
- r_{ij} is the distance between the centres of robot i and robot j where $i, j \leq n$,
- $d_{o \rightarrow j}$ is the distance between the centres of robot j and its closest obstacle,
- $\sigma(V)$ is the standard deviation of the n robots' linear velocities.

3.1 Decision Variables

The goal of this optimization problem is to find paths K that is presented by a $[1 \times n]$ array for the n robots to follow from the start point S to the end goal G . The paths will consist of a

number of points N_p for each robot. Each point is a tuple containing the x position and the y position of the i_{th} robot $(x, y)_i$.

3.2 Objective Functions

To achieve the desired behavior this section presents the There are four objective functions in our problem. The mobile robots try to reach our final destination represented by point G whilst flocking.

The first one is to keep the distance between each two robots around a certain value r_f . This addresses the cohesion and separation part of the flocking problem.

$$\operatorname{argmin}_K \sum_{i=1}^n |r_{ij} - r_f| \quad (3.1)$$

The second objective function deals with matching the velocities of the robots. This is related to alignment. This is done by minimizing the standard deviation of the robots' velocities.

$$\operatorname{argmin}_K \sigma(V) \quad (3.2)$$

The third objective function is to avoid obstacles. The obstacles are defined as circles regardless of their shapes with a radius r_o .

$$\operatorname{argmax}_K \sum_{i=1}^n |d_{o \rightarrow j} - (r_{oi} + r_{bi}) - c| \quad (3.3)$$

The fourth objective function is minimizing the change in the steering angles of the mobile robots to avoid sudden changes in trajectory. θ_i defines the change in the angle of the i_{th} robot.

$$\operatorname{argmin}_K \sum_{i=1}^n \Delta\theta_i \quad (3.4)$$

The cost function is defined as summing all previous elements elements multiplied by four different weights.

$$cost = w_1 * \sum_{i=1}^n |r_{ij} - r_f| + w_2 * \sigma(V) + w_3 * \sum_{i=1}^n |d_{o \rightarrow j} - (r_o + r_b) - c| + w_4 * \sum_{i=1}^n \Delta\theta_i \quad (3.5)$$

3.3 Constraints

The first constrain is to stay inside the map.

$$0 < P_{ix} < L \quad \forall i \quad (3.6)$$

$$0 < P_{iy} < W \quad \forall i \quad (3.7)$$

The second constraint is that the velocity of each robot must not exceed the maximum velocity V_{max}

$$V_i < V_{max} \quad \forall i \quad (3.8)$$

The third constraint is the steering angle. The steering angle is not allowed to exceed a maximum limit δ_f

$$\theta_i < \delta_f \quad \forall i \quad (3.9)$$

3.4 Mobile Robot Kinematics

The differential robot that will be used consists of two back wheels that are responsible for controlling the robot motion and one passive front wheel for balancing. Since the robot can move freely in a 2D-plane, it has 3 DOF which requires 3 independent parameters to describe its location. This kinematic model will consider three variables and their rate of change:

- θ : The rotation angle of the differential robot relative to the $+ve$ x-axis
- (x, y) : The location of a fixed point on the differential robot relative to a fixed frame on the map. In this model, this point is the middle point between the two back wheels.

Since the robot performs general plane motion, its velocity can be described by two velocity vectors, the velocity vector of the chosen fixed point v_T in addition to the whole robot angular velocity vector ω (figure 3.1a).

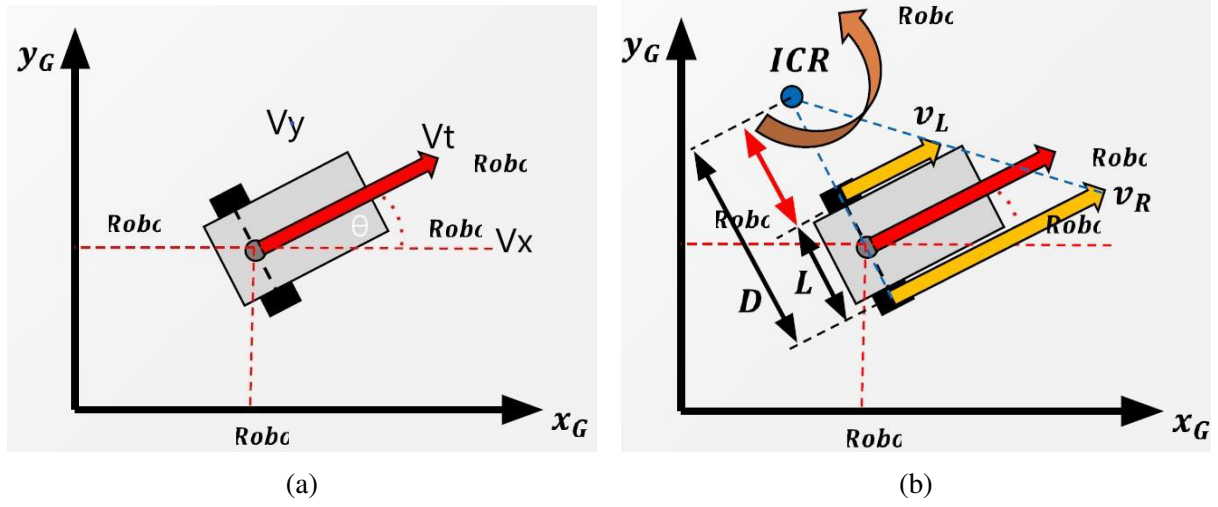


Figure 3.1: Visual kinematic analysis of the differential mobile robot

The velocity of the fixed point can be obtained from the velocity of the right and left wheels (figure 3.1b).

$$v_T = \frac{v_R + v_L}{2} \quad (3.10)$$

$$v_x = v_T \cos \theta, \quad v_y = v_T \sin \theta$$

The velocity of the right and left wheels can be obtained from the angular velocity of each wheel (figure 3.1b).

$$v_R = \omega_R \times r \quad (3.11)$$

$$v_L = \omega_L \times r$$

Where ω_R, ω_L, r are the angular velocity of the right wheel, the angular velocity of the left wheel and the radius of the wheel, respectively. By substituting from 3.11 in 3.10:

$$v_T = \frac{\omega_R \times r + \omega_L \times r}{2} = \frac{r}{2}(\omega_R + \omega_L) \quad (3.12)$$

$$v_x = \frac{r \cos \theta}{2}(\omega_R + \omega_L) \quad (3.13)$$

$$v_y = \frac{r \sin \theta}{2}(\omega_R + \omega_L) \quad (3.14)$$

Equations (3.13) and (3.14) describe the x and y components of the velocity vector for the fixed point. The angular velocity of the whole robot ω can be obtained from the following

equations that are obtained from figure 3.1b.

$$v_R = \omega \times D \quad , \quad v_L = \omega \times (D - L)$$

Solving the two equations for ω :

$$\omega = \frac{v_R - v_L}{L} = \frac{r}{L}(\omega_R - \omega_L) = \dot{\theta} \quad (3.15)$$

Equations (3.13), (3.14) and (3.15) are used to control the motion of the robot.

Chapter 4

Simulated Annealing

In this chapter, we are going to implement the simulated annealing algorithm on the chosen problem which is multi robot path planning for dynamic formation applications.

4.1 Problem Specific Code

Before starting to implement the SA algorithm the foundation of our problem is laid where we define a number of matlab structs that carry most of our problem definitions these structs could be found in the MainFile.m file. The main highlights are defining the "robots" which are a number of differential drive kinematic objects. After which some simple pure pursuit controllers are introduced with their parameters. The reason behind needing such setup is that this problem can't be solved without having a time context for the robots in addition to their positions. Another reason is needing to know the robots velocities following the generated paths to check for alignment.

4.2 SA Algorithm

The SA algorithm described in the lecture was implemented. The act of saving the best solution was implemented for personal reference only it is not used in case studies only the last solution is considered. Generating waypoints and simulating kinematics are problem specific codes that needed to be added before the cost function could be implemented.

4.3 Flow Chart

A flow chart shows how the SA algorithm works for our problem, shown in Fig 5.1.

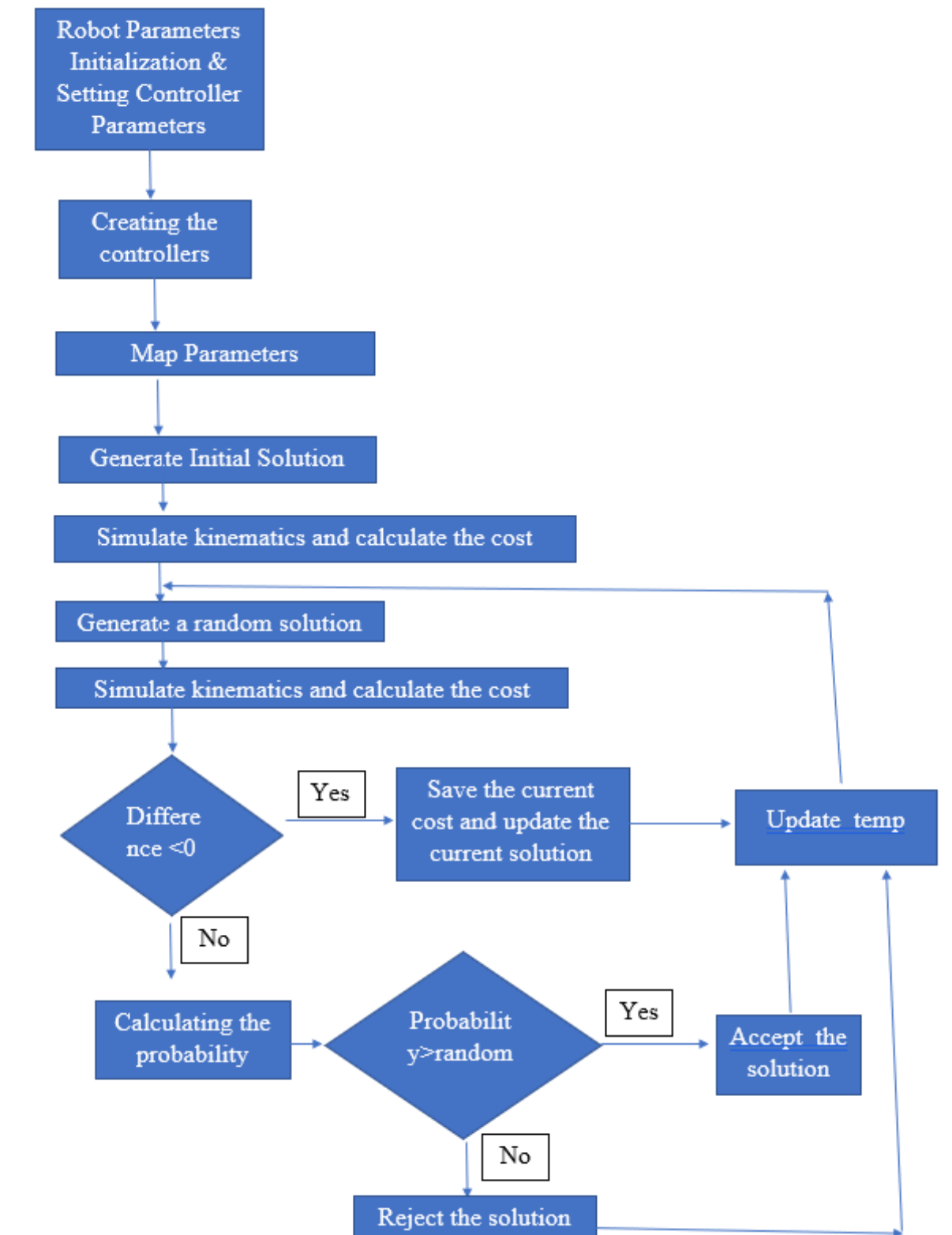


Figure 4.1: Flowchart of SA Algorithm

Chapter 5

Genetic Algorithm

In this chapter, we are going to implement genetic algorithm on the chosen problem which is multi robot path planning for dynamic formation applications.

5.1 Problem Specific Code

based on the foundations of our problem that was defined in the previous chapters we added a new object which is the GA. The GA contains all the information about the population. Each chromosome has n number of genes where n is the number of Robots the genes are the way points required for each robot to reach the its desired goal as it is a (*Way – points* X 2) matrix which contains the XY coordinates for each chosen point we simulate the points to calculate the cost function to each chromosome and assign it to each member as its fitness. The method of the cross-over is the Whole arithmetic and for the parents selection we can choose between four different methods of selection: Roulette wheel selection, Stochastic universal sampling, Rank selection and Tournament selection. For survivors selection we implemented two different methods: fitness based selection and age based selection. For case studying purpose we defined all possible ways and we can select between them to asses the performance of each method finally the mutation is done by generating a new random solution for a random gene.

5.2 GA Algorithm

The GA algorithm is coded based on the work explained in the Book "Meta-heuristic and evolutionary algorithms for engineering optimization by Bozorg-Haddad, Omid Loaiciga, Hugo A. Solgi, Mohammad" in the next section a state flow chart will give a better insight about how the algorithm operates.

5.3 Flow Chart

A flow chart shows how the GA algorithm works for our problem, shown in Fig 5.1.

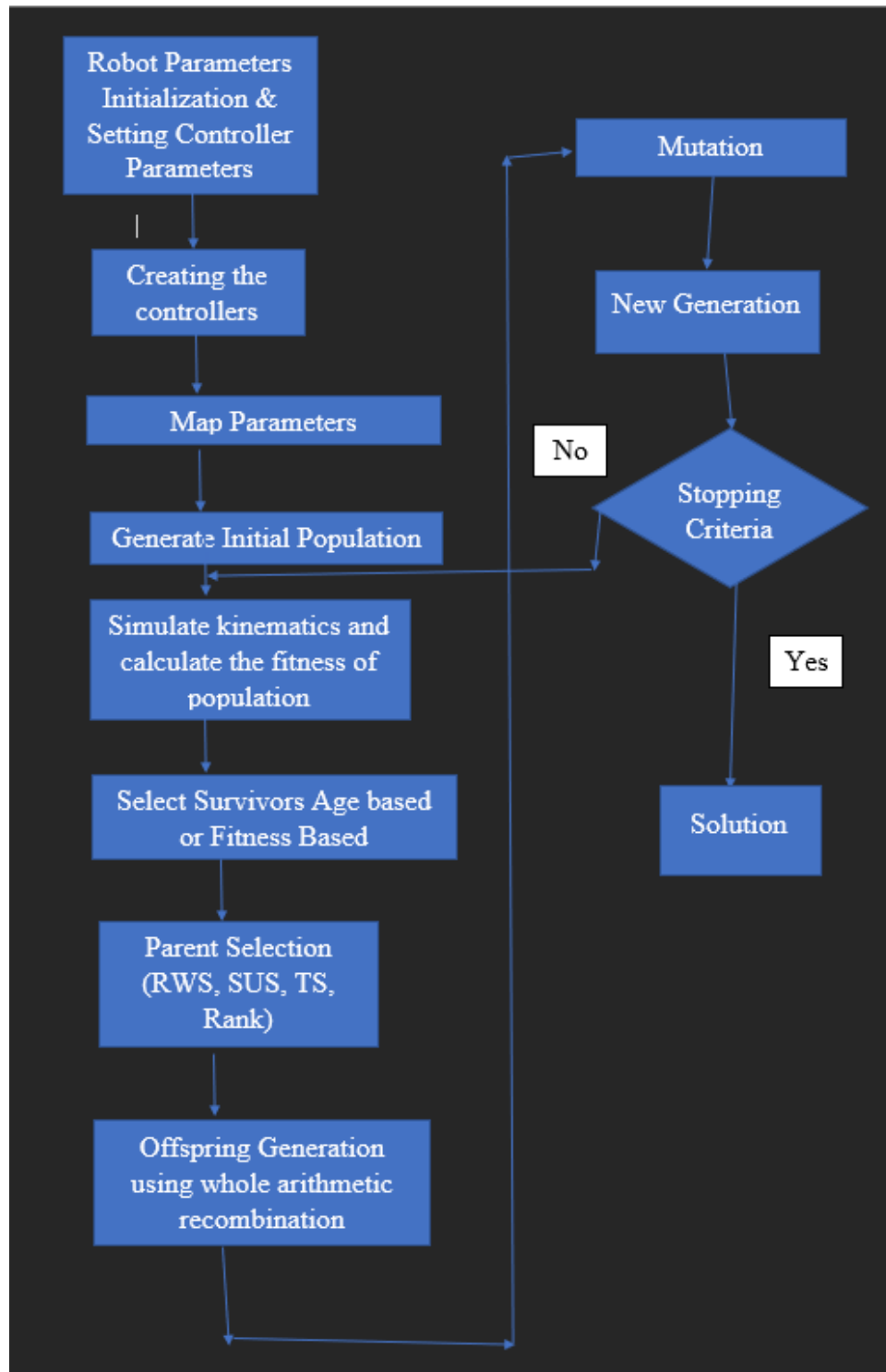


Figure 5.1: Flowchart of GA Algorithm

Chapter 6

Results

6.1 SA

SA results in general are not satisfactory especially with higher number of waypoints and robots. This is mainly attributed to the fact that the search space is very large.

The capabilities of trajectory-based methods is not enough to find a sub-optimal solution. Here only 2 runs will be discussed they use the same SA parameters with the exception of a different cooling schedule.

The first run to discuss was run with the following SA parameters and a linear cooling schedule 4.999.

Table 6.1: Linear Run Parameters

Parameter	Value
Max number of iterations	1000
Initial Temperature	500
Final Temperature	1
Randome Range	10
alpha	$(\text{Initial Temp} - \text{Final Temp}) / \text{Max num of iterations}$
beta	0.8
Linear Cooling	True
Check Probability	True

To briefly explain the graphs. The leftmost graph is the final trajectory chosen by the algorithm at the 1000th iteration. The center graph is the cost of the solution at each iteration. Finally we have the temperature. It can be seen that the algorithm couldn't successfully satisfy

the soft constrain placed on obstacle avoidance. With a linear cooling schedule -unlike what we will see with a geometric one- there is a lot of jumps to worse solutions this is because the temperature remains relatively high throughout the run time. This provides a good amount of exploration in the early stages of finding a solution but not enough exploitation.

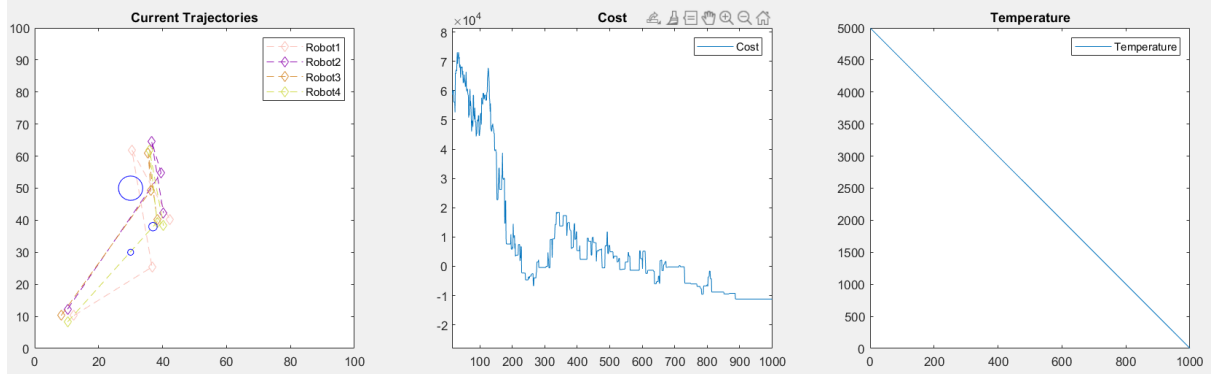


Figure 6.1: Results of running the SA with linear cooling schedule

In the following graphs are the main terms that constitute the cost function. The final using the linear cooling scheduled solution was relatively good in keeping the distance around the fixed constant r_f while both runs struggled in keeping a similar velocity across all robots. Both runs couldn't adhere to the obstacle avoidance soft constraint which requires further experimentation with the weights to put higher emphasis on this aspect of the cost function.

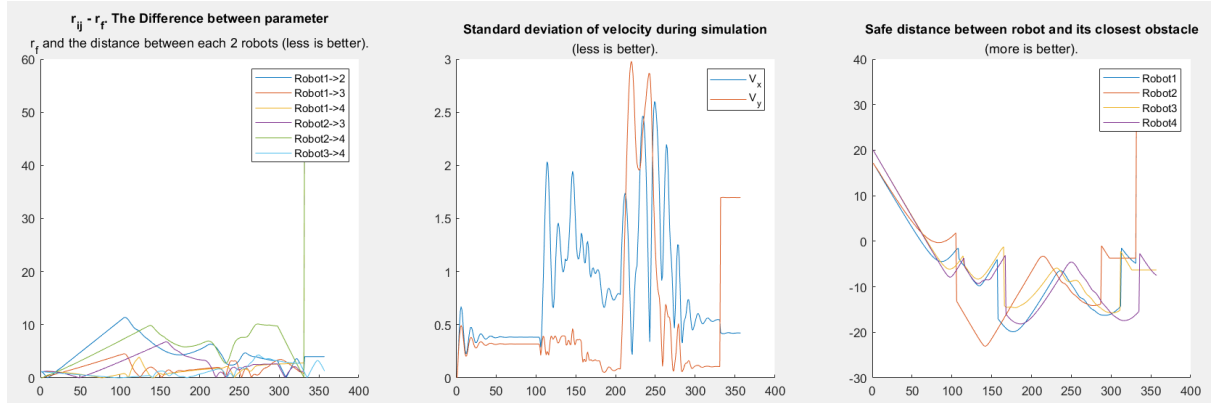


Figure 6.2: Results of running the SA with linear cooling schedule

Table 6.2: Geometric Run Parameters

Parameter	Value
Max number of iterations	1000
Initial Temperature	500
Final Temperature	1
Randome Range	10
alpha	(Initial Temp - Final Temp) / Max num of iterations
beta	0.8
Linear Cooling	Flase
Check Probability	True

The geometric cooling schedule explores much less compared to the linear cooling schedule. The alignment in this run is better than the one achieved using linear cooling this can't be attributed to a specific characteristic of the geometric cooling schedule.

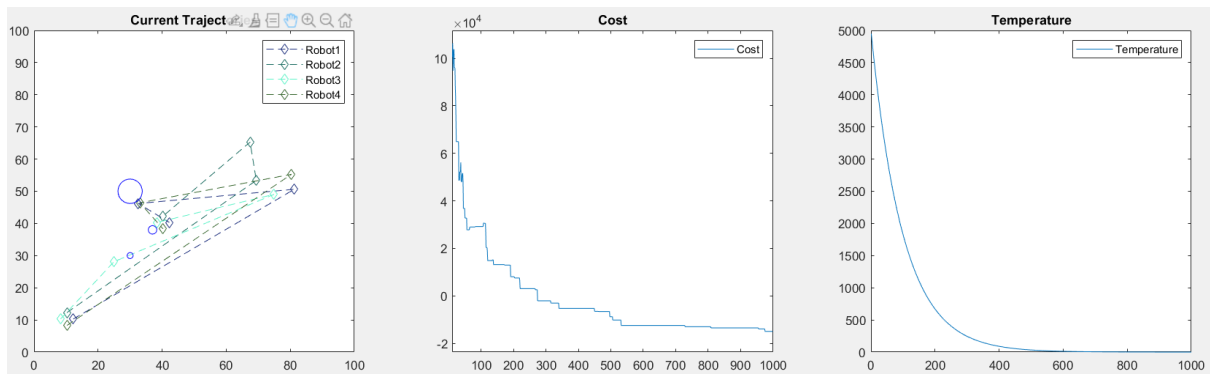


Figure 6.3: Results of running the SA with geometric cooling schedule

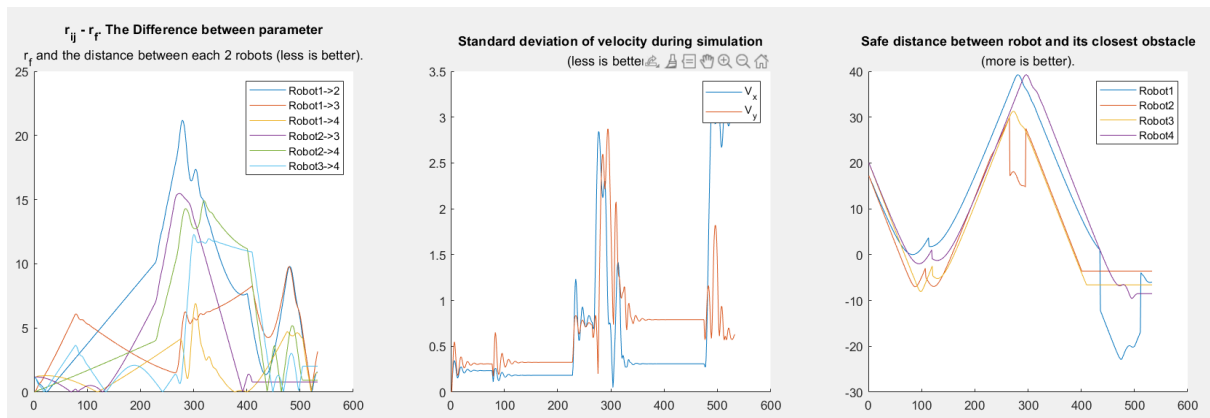


Figure 6.4: Results of running the SA with linear cooling schedule

6.2 GA

The results obtained from GA algorithm are obtained by running the code using population of 20 running for 50 generations we are plotting the results of our problem parameters used in the cost function in different runs with changing the method of selecting the parents using $\alpha = 0.7$ for the cross over

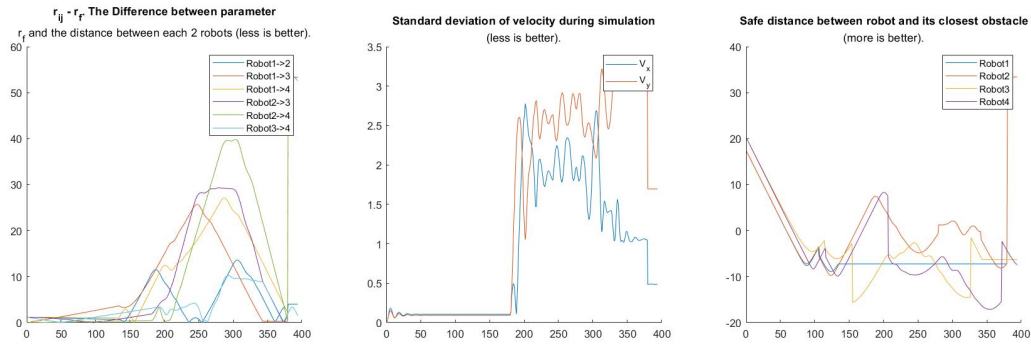


Figure 6.5: Results of running the GA with Roulette wheel selection

the difference between the robots is small in the beginning of the operation as well as the deviation of the velocity however in the middle of the operation when facing obstacles the robots starts drafting to get around the obstacles

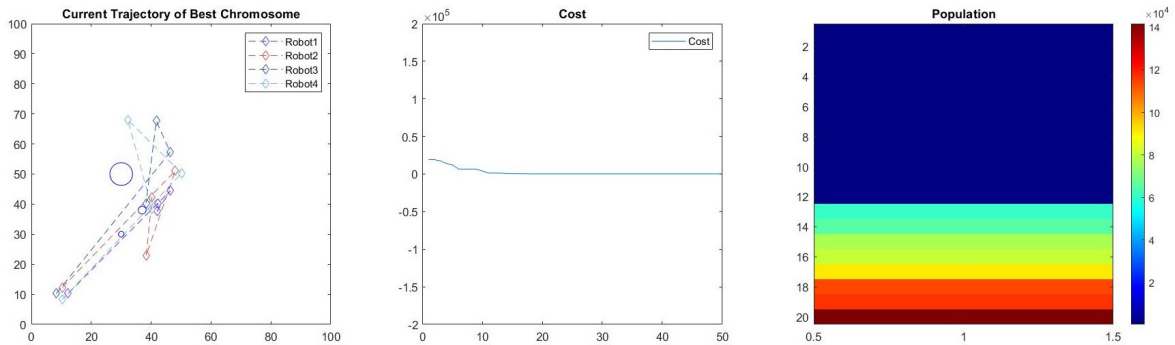


Figure 6.6: Results of running the GA and visualizing on a map for the Roulette wheel selection

the cost of the solution tends to converge fast in the GA and the paths tend to be sub-optimal

based on the formulation

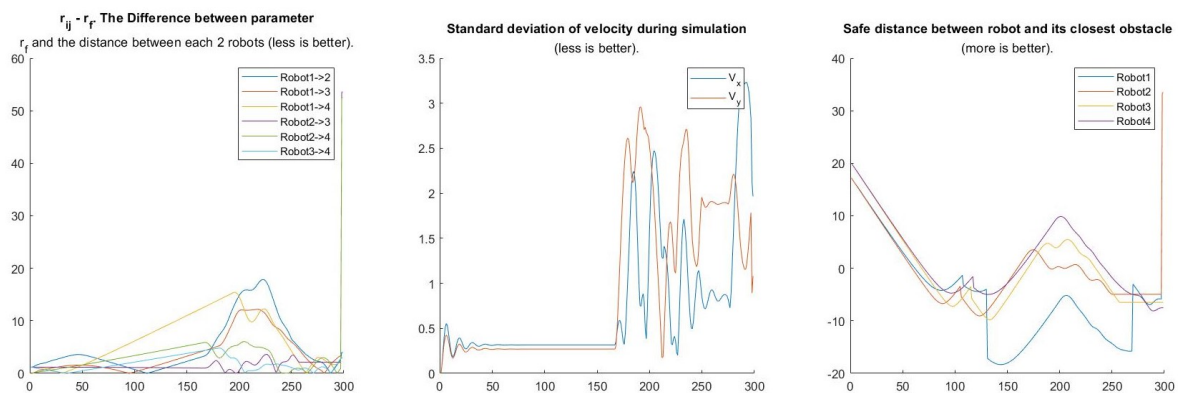


Figure 6.7: Results of running the GA using Tournament selection

The results of the Tournament selection has less distances between the robots than the Roulette selection which also gives faster cost convergence

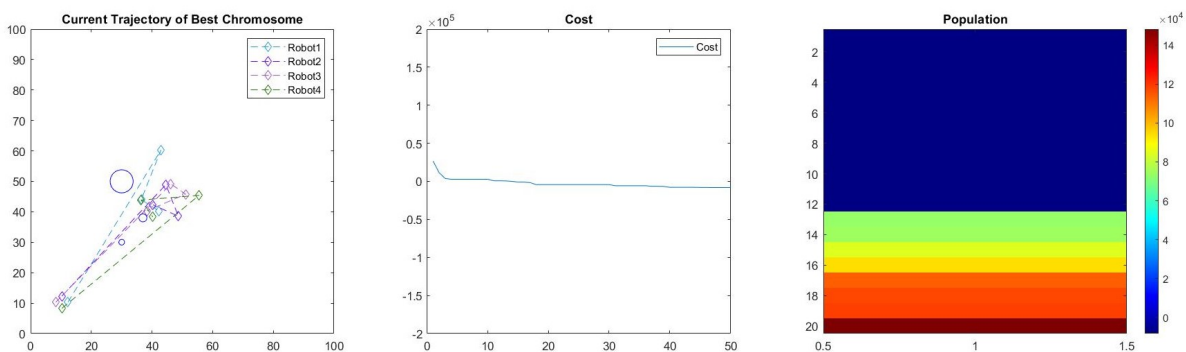


Figure 6.8: Results of running the GA and visualizing on a map for the Tournament selection

In the rank selection the results seems to be bad when visualized in a map as the robots go to explore points that are far from the goal then return to the goal however in terms of the distances between the robots it does not have extreme point where the robots are far from each others such as in the case of the Roulette wheel selection.

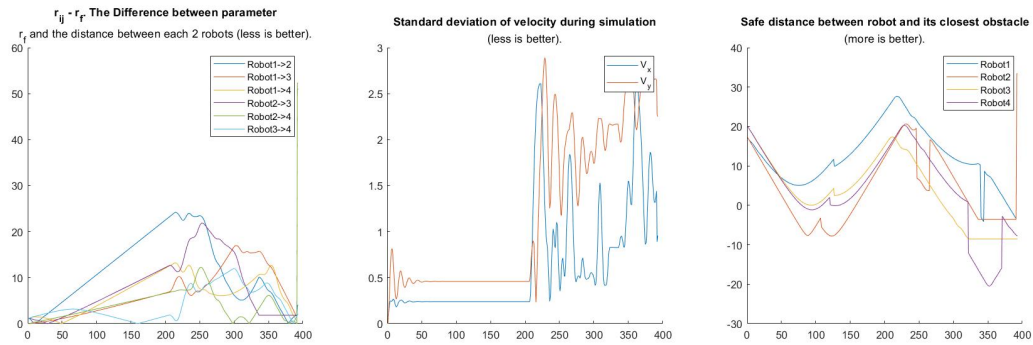


Figure 6.9: Results of running the GA With Rank selection

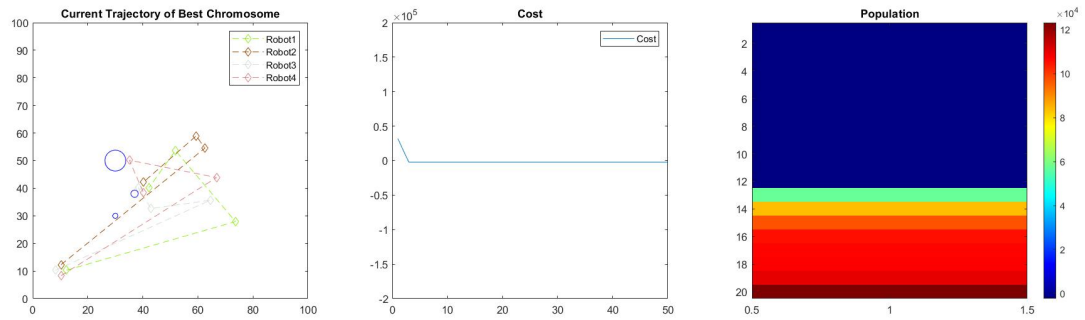


Figure 6.10: Results of running the GA and visualizing on a map for the Rank selection

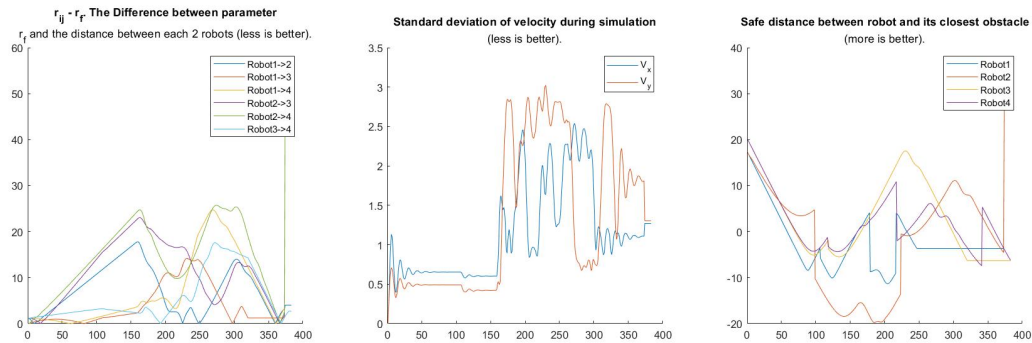


Figure 6.11: Results of running the GA and visualizing on a map for the SUS selection

The SUS selection has the worst results of all the selection methods as the fluctuation in the distance parameter is high with a path that has the robots to get far away from each other and the final goal

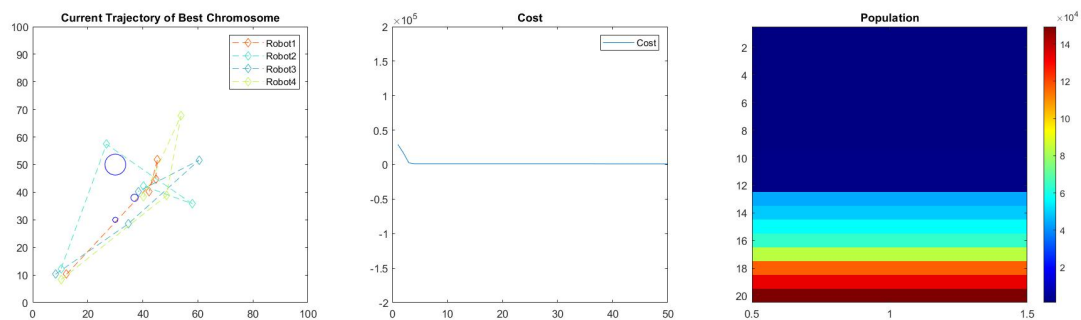


Figure 6.12: Results of running the GA and visualizing on a map for the Rank SUS selection

Overall the Tournament selection seems to be the best of the four options in terms of faster convergence, shorter path and less violation to the cost function all things considered.

In conclusion, GA is considered better than SA in our particular problem as it explores wider range of possibilities, have faster convergence and better path

Chapter 7

Conclusion

Chapter 8

Future Work

Appendix

Bibliography

- [1] Tatsuya Ibuki, Sean Wilson, Junya Yamauchi, Masayuki Fujita, and Magnus Egerstedt. Optimization-based distributed flocking control for multiple rigid bodies. *IEEE Robotics and Automation Letters*, 5(2):1891–1898, 2020.
- [2] Hehui Zheng, Jacopo Panerati, Giovanni Beltrame, and Amanda Prorok. An adversarial approach to private flocking in mobile robot teams. *IEEE Robotics and Automation Letters*, 5(2):1009–1016, 2020.
- [3] Lifeng Zhou and Shaoyuan Li. Distributed model predictive control for multi-agent flocking via neighbor screening optimization. *International Journal of Robust and Nonlinear Control*, 27(9):1690–1705, 2017.
- [4] E Khodayari, V Sattari-Naeini, and M Mirhosseini. A gravitational search algorithm-based single-center of mass flocking control for tracking single and multiple dynamic targets for parabolic trajectories in mobile sensor networks. *Journal of AI and Data Mining*, 6(1):207–217, 2018.
- [5] Ibrahim Hamad and Mohammad Hasan. A review: On using aco based hybrid algorithms for path planning of multi-mobile robotics. 2020.
- [6] Huaxin Qiu and Haibin Duan. A multi-objective pigeon-inspired optimization approach to uav distributed flocking among obstacles. *Information Sciences*, 509:515–529, 2020.
- [7] Huaxin Qiu and Haibin Duan. Pigeon interaction mode switch-based uav distributed flocking control under obstacle environments. *ISA transactions*, 71:93–102, 2017.
- [8] Gábor Vásárhelyi, Csaba Virágh, Gergő Somorjai, Tamás Nepusz, Agoston E Eiben, and Tamás Vicsek. Optimized flocking of autonomous drones in confined environments. *Science Robotics*, 3(20), 2018.
- [9] Seunghan Lim, Yeongho Song, Joonwon Choi, Hyunsam Myung, Heungsik Lim, and Hyondong Oh. Decentralized hybrid flocking guidance for a swarm of small uavs. In *2019 Workshop on Research, Education and Development of Unmanned Aerial Systems (RED UAS)*, pages 287–296. IEEE, 2019.

- [10] Li Ma, Weidong Bao, Xiaomin Zhu, Meng Wu, Yuan Wang, Yunxiang Ling, and Wen Zhou. G-flocking: Flocking model optimization based on genetic framework. *ArXiv*, abs/1907.11852, 2019.